



Solemne 3

INDICACIONES

- Esta Solemne se realiza de forma individual.
- La solemne debe ser rendida en la plataforma <http://codetrack.cl/>
- Para entrar debe escribir su correo institucional y como password su rut (solo números).
- Lea cada pregunta con atención y programe en la plataforma.
- No se permite el uso de dispositivos electrónicos como celulares, relojes inteligentes, tablets, etc. Quien sea sorprendido/a con alguno de ellos, se le evaluará con la nota mínima.
- Cualquier consulta sobre la plataforma debe ser resuelta con los profesores a cargo.
- Puede usar su cheat sheet impreso y entregado en clases.

Problemas

1. Limpieza y resumen de un catálogo estelar con pandas.

Enunciado

Un catálogo pequeño de estrellas fue armado desde distintas fuentes y guardado en un archivo CSV. Antes de comparar las estrellas, se debe leer la tabla, inspeccionarla, eliminar mediciones incompletas y crear una columna física derivada.

El archivo CSV se encuentra en la siguiente ruta:

```
1 url = (  
2     "https://raw.githubusercontent.com/jperaltac/pcfi161/"  
3     "main/Solemnes/2026/S3/catalogo_estrellas_s3.csv"  
4 )
```

Realice las siguientes operaciones:

- Importe `numpy`, `pandas` y `matplotlib.pyplot`. Lea el archivo CSV usando `pd.read_csv(url)` y guárdelo en un DataFrame llamado `estrellas`.
- Inspeccione la tabla mostrando las primeras filas, dimensiones, tipos de datos y cantidad de valores faltantes por columna.
- Cree un DataFrame llamado `estrellas_limpias` eliminando solo las filas que tengan datos faltantes en `temperatura_K`, `radio_Rsol`, `distancia_pc` o `magnitud_V`. Use `copy()` al final.
- En `estrellas_limpias`, cree la columna `luminosidad_rel` usando

$$L_{\text{rel}} = R^2 \left(\frac{T}{5778} \right)^4,$$

donde R es `radio_Rsol` y T es `temperatura_K`.

- Agrupe `estrellas_limpias` por tipo y guarde ese grupo en una variable.
- Antes de calcular estadísticas, cree una copia de `estrellas_limpias` sin la columna `nombre`. Agrupe esa nueva tabla por tipo y calcule cuántas estrellas hay en cada tipo, el promedio por tipo y la desviación estándar por tipo. Imprima estos resultados.
- Haga un gráfico de dispersión simple usando dos columnas del DataFrame. Grafique `temperatura_K` en el eje horizontal y `luminosidad_rel` en el eje vertical. Agregue etiquetas, título y grilla.

2. Ordenamiento y búsqueda de períodos orbitales.

Enunciado

Un equipo observa exoplanetas y registra sus períodos orbitales en días:

`periodos = [12.4, 3.1, 8.7, 25.0, 1.9, 14.2, 6.5, 18.3].`

Utilice los algoritmos entregados a continuación. No debe escribir otro algoritmo de ordenamiento ni otro algoritmo de búsqueda. En una parte del ejercicio se le pedirá modificar `bubble_sort` para contar comparaciones e intercambios:

```
1 def bubble_sort(lista):
2     n = len(lista)
3     for i in range(n - 1):
4         for j in range(n - 1 - i):
5             if lista[j] > lista[j + 1]:
6                 lista[j], lista[j + 1] = lista[j + 1], lista[j]
7     return lista
8
9 def binary_search(lista_ordenada, objetivo):
10     low, high = 0, len(lista_ordenada) - 1
11
12     while low <= high:
13         mid = (low + high) // 2
14
15         if lista_ordenada[mid] == objetivo:
16             return mid
17         elif objetivo < lista_ordenada[mid]:
18             high = mid - 1
19         else:
20             low = mid + 1
21
22     return -1
```

Realice las siguientes operaciones:

- (a) Defina la lista `periodos`.
- (b) Modifique la función `bubble_sort` entregada para que también cuente cuántas comparaciones entre elementos realiza y cuántos intercambios hace. La función modificada debe devolver tres valores: la lista ordenada, el número de comparaciones y el número de intercambios. No modifique `binary_search`.
- (c) Ordene los períodos de menor a mayor usando la función `bubble_sort` modificada. Debe ordenar una copia de la lista original, no la lista original directamente.
- (d) Imprima la lista original, la lista ordenada, el número de comparaciones y el número de intercambios.
- (e) Determine el período mínimo y el período máximo usando la lista ordenada.
- (f) Use `binary_search` para buscar el período 14.2 en la lista ordenada. Imprima su posición si fue encontrado.
- (g) Use `binary_search` para buscar el período 10.0. Si no aparece, imprima un mensaje claro indicando que no fue encontrado.

3. Ajuste cuadrático y estimación de un umbral.

Enunciado

En una calibración de laboratorio se mide la respuesta R de un detector para distintas dosis d . Los datos medidos son:

$$d = [0, 1, 2, 3, 4, 5, 6],$$

$$R = [1.2, 2.0, 4.1, 6.9, 10.8, 16.2, 22.1].$$

Se desea construir un modelo cuadrático

$$R(d) = ad^2 + bd + c$$

y usarlo para estimar cuándo la respuesta alcanza un umbral de seguridad

$$R = 12.$$

Realice las siguientes operaciones:

- (a) Importe `numpy` y `matplotlib.pyplot`. Defina los arreglos `d` y `R` usando `np.array`.
- (b) Use `np.polyfit(d, R, deg=2)` para obtener los coeficientes a , b y c .
- (c) Calcule los valores ajustados sobre los mismos datos medidos y guárdelos en `R_fit`.
- (d) Calcule los residuos y el coeficiente R^2 :

$$\text{residuos} = R - R_{\text{fit}},$$

$$SS_{\text{res}} = \sum_i (R_i - R_{\text{fit},i})^2, \quad SS_{\text{tot}} = \sum_i (R_i - \bar{R})^2,$$

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}.$$

- (e) Use `np.roots` para resolver cuando el modelo ajustado alcanza $R = 12$. Para ello debe resolver

$$ad^2 + bd + c - 12 = 0.$$

- (f) Filtre las raíces: conserve solo soluciones reales dentro del rango medido, es decir, entre 0 y 6.
- (g) Grafique los datos medidos con `plt.scatter`, el ajuste cuadrático con `plt.plot` usando una grilla fina para la dosis, y la línea horizontal $R = 12$. Para esta grilla fina, cree un arreglo con `np.linspace` para la variable independiente. Agregue etiquetas, leyenda y active la grilla del gráfico.
- (h) Imprima a , b , c , R^2 y la dosis estimada para alcanzar el umbral. Agregue una frase que explique por qué no basta con tomar cualquier raíz entregada por `np.roots`.